

KEY MATRIX — Binary Analysis / Forensics

Category: Forensics

Subcategory: Binary Analysis / ELF Analysis

Difficulty: Medium

Challenge File: challenge.bin

Flag Format: OBV{...}

Flag: OBV{K3Y_M4TR1X_COLLAPSED}

DESCRIPTION

A suspicious Linux ELF binary was recovered during a forensic investigation. The binary does not contain the flag in readable plaintext. The investigator must identify the relevant ELF section, recover the hidden byte sequence, find the clue revealing the transformation key, and decode the payload.

*The only explicit clue provided to the player is: **OBV{...}***

OBJECTIVE

Analyze challenge.bin as a forensic binary artifact and recover the hidden flag.

INTENDED INVESTIGATION

ZIP → challenge.bin → Identify ELF → Inspect ELF sections → Find .note.cache → Inspect/extract the section → Find "SYNC CHANNEL: 5A" → Use 0x5A as the XOR key → Decode the payload → OBV{K3Y_M4TR1X_COLLAPSED}

1. EXTRACT THE CHALLENGE

The player receives:

Shell

```
unzip Binary_Analysis_Challenge.zip
```

Expected Output:

None

```
Archive:  Binary_Analysis_Challenge.zip
  inflating: challenge.bin
```

Why?

The challenge is distributed as an archive. The actual forensic artifact is

challenge.bin, so the first step is to extract it before analysis.

2. IDENTIFY THE FILE

Shell

```
file challenge.bin
```

Expected result:

None

```
challenge.bin: ELF 64-bit LSB pie executable, x86-64, version  
1 (SYSV), dynamically linked, interpreter  
/lib64/ld-linux-x86-64.so.2, stripped
```

Why?

The `file` command identifies the basic file type. Here it reveals that the artifact is an ELF executable for Linux.

The binary should be treated as a static-analysis artifact. There is no need

to execute it to solve the challenge.

3. INSPECT THE ELF SECTION TABLE

Shell

```
readelf -S challenge.bin
```

Why?

`readelf -S` lists the sections contained in an ELF file. The custom `.note.cache` section is unusual compared with the standard executable sections and is therefore worth investigating.

4. INSPECT THE SUSPICIOUS SECTION

Shell

```
objdump -s -j .note.cache challenge.bin
```

Expected Output:

None

```
Contents of section .note.cache:
```

```
0000 15180c21 11690305 176e0e08 6b020519
```

```
0010 1516161b 0a091f1e 27
```

Why?

objdump displays the raw contents of a selected ELF section. The bytes are not readable ASCII, suggesting encoded or encrypted data.

5. SEARCH FOR HUMAN-READABLE CLUES

Shell

```
strings -a challenge.bin
```

Why?

strings extracts printable character sequences from the binary. The "SYNC CHANNEL: 5A" text provides a hexadecimal value that can be tested as the transformation key for the suspicious byte sequence.

6. EXTRACT THE HIDDEN SECTION

Shell

```
objcopy --dump-section .note.cache=payload.bin challenge.bin
```

Why?

Separating the section into payload.bin makes the byte sequence easier to analyze independently from the ELF container.

7. DECODE THE PAYLOAD

Shell

```
python3 - <<'PY'  
from pathlib import Path  
data = Path("payload.bin").read_bytes()  
key = 0x5A  
decoded = bytes(b ^ key for b in data)  
print(decoded.decode())  
PY
```

8. MANUAL XOR VERIFICATION

The first ciphertext byte is:

15

XOR with:

5A

gives:

15 XOR 5A = 4F

Hex 4F is ASCII:

O

Continuing the same operation over the entire payload produces:

OBV{K3Y_M4TR1X_COLLAPSED}

FINAL FLAG

OBV{K3Y_M4TR1X_COLLAPSED}

COMPLETE COMMAND SEQUENCE

Shell

```
unzip Binary_Analysis_Challenge.zip
ls -lh
file challenge.bin
readelf -S challenge.bin
objdump -s -j .note.cache challenge.bin
strings -a challenge.bin
objcopy --dump-section .note.cache=payload.bin challenge.bin
xxd payload.bin
```